```
Output:
<!DOCTYPE html>
<html
  lang="en"
  data-color-mode="dark" data-dark-theme="dark"
  ...

resource(5) of type (stream)
resource(5) of type (Unknown)
```

The first argument to `fopen()` above is a URL (Uniform Resource Locator), as I'm sure you're aware. `fopen()` is able to parse this and work out a few important things: where to open the stream (by resolving the IP address of `github.com`), which TCP port to use (`443` for `https`), and a few other things (the protocol handshaking required for HTTP). Be aware that this behavior is controlled by the `allow_url_fopen` directive in your `.ini` files, though it is usually enabled by default. Chapter 11, "Programming for the Internet," covers this in more detail.

Once the connection is established, the file handle is ready for use and assigned to the `$handle` variable. Then a simple `while` loop (see Chapter 4, "Logic and Control," for more info) reads from the handle a line at a time. In this case, we're simply printing out each line; a real-world program might be looking for patterns or modifying each line somehow… All of that kind of goodness is dealt with in Chapter 8, "Strings." After we're done with the handle (`feof()` tests the handle for EOF on every iteration of the loop), we call `fclose()` – you can see from the output that after this call `$handle` is no longer pointing to a stream.

One thing to note: handling input this way does not consume much memory, because we don't need to load the entire contents of `https://github.com` into memory before we start to print it out. Take a look at the following:

```
$handle = fopen('https://github.com', 'r');
while (!feof($handle)) {
    print fgets($handle);
}
var_dump(memory_get_usage());

Output:
...
int(407792)
```